

AI-Based Cybersecurity Threat Risk Analyzer

INDIVIDUAL IKS ACADEMIC PROJECT REPORT

Student Name: [INSERT STUDENT NAME]

Roll Number: [INSERT ROLL NUMBER]

Class / Stream: T.Y. B.Sc. IT

College: [INSERT COLLEGE NAME]

Faculty / Guide: [INSERT FACULTY NAME]

Academic Year: 2025–2026

Subject: IKS / Applied AI

Project Summary

A Streamlit application that combines LangChain + Google Gemini for natural-language cybersecurity incident understanding with a genuine Mamdani fuzzy inference system for the final numerical risk score.

Abstract

The project presents an AI-assisted cybersecurity risk analyzer. A user describes an incident in natural language. LangChain with Gemini extracts structured cybersecurity factors such as likelihood, impact, exposure and data sensitivity. These validated values are passed to a fuzzy inference system. The fuzzy engine performs fuzzification, rule evaluation, aggregation and centroid defuzzification to calculate a risk score from 0–100. The LLM explains the result and provides defensive recommendations; it does not calculate the final risk score.

Introduction

Cybersecurity incidents are often reported informally and may contain uncertainty. A fixed threshold system can create abrupt changes, while a pure LLM can produce inconsistent numerical judgments. This project combines language understanding with transparent fuzzy reasoning.

Problem Statement

Build an educational decision-support application that converts a natural-language security incident into validated risk factors and produces an explainable fuzzy risk score that can help prioritize defensive attention.

Objectives

- Extract structured cybersecurity factors from natural-language incident descriptions.
- Implement a genuine fuzzy inference system with membership functions, rules and centroid defuzzification.
- Keep the LLM separate from the final numerical risk decision.
- Provide an accessible Streamlit interface with explanations and defensive recommendations.
- Demonstrate secure secret handling and automated tests.

Scope

The project covers natural-language incident analysis, fuzzy risk scoring, defensive recommendations, visualisation and educational demonstration. It does not perform offensive actions, real-time SOC monitoring, SIEM integration or production-grade risk certification.

Technologies

Python • Streamlit • LangChain Core • langchain-google-genai • Google Gemini 3.8 Flash • scikit-fuzzy • NumPy • Matplotlib • Pydantic • python-dotenv • pytest.

IKS Concept / Source

The project makes a conceptual connection with the Arthaśāstra tradition of Indian statecraft and security thought. The Arthaśāstra is a classical Indian treatise associated with Kauṭilya and contains material concerning governance, state security, intelligence and surveillance.

Modern IT/AI Connection

The project translates the broad security-oriented idea of identifying threats, considering their potential consequences and supporting defensive action into a modern cybersecurity setting. Natural-language incident information is interpreted using an LLM, while fuzzy rules provide a transparent risk-assessment mechanism.

Important Academic Clarification

This is a conceptual connection, not a claim that the ancient text contains modern cybersecurity, machine learning or fuzzy-logic methods. LangChain, Gemini, scikit-fuzzy and Streamlit are modern technologies.

Source used for this connection

Cambridge University Press scholarship on ancient Indian security and the Arthaśāstra, including discussion of clandestine operations and surveillance in the context of state security. See References on Page 9.

Faculty check

The student should confirm that this IKS source/connection is acceptable to the faculty/guide before final submission, because the college instructions make the IKS connection mandatory.

Workflow

Natural-language incident → LangChain/Gemini factor extraction → Pydantic validation → Fuzzy fuzzification → 17 Mamdani rules → aggregation → centroid defuzzification → risk score/category → LLM explanation + defensive recommendations → Streamlit display.

Four fuzzy inputs

Threat Likelihood, Potential Impact, Exposure and Data Sensitivity, each represented on a 0–100 universe.

Fuzzy output

Cybersecurity Risk from 0–100, with Low, Medium, High and Critical output membership functions.

Method

The LLM performs language understanding and structured extraction. Pydantic validates the extracted values. The fuzzy engine alone performs the numerical risk calculation. The explanation chain receives the already-computed score and explains it rather than replacing it.

Major modules

Module	Role
app.py	Streamlit UI and visualisation
llm_service.py	LangChain chains, Gemini calls and output parsing
fuzzy_engine.py	Membership functions, rules, aggregation and defuzzification
risk_analyzer.py	Pipeline orchestration
validators.py	Pydantic validation
prompts.py	Prompt templates
sample_cases.py	Demonstration scenarios

Security

No API key is hard-coded. LLM outputs are parsed and validated. The application is defensive-only. Missing or malformed LLM output is handled with safe fallback behaviour.

Demonstration scenarios

Scenario	Expected risk band
Phishing credential compromise	Critical / High
Suspicious login activity	High
Malware alert on workstation	High
Unusual file access	High
Possible sensitive-data exposure	Medium / High

Application result

The UI displays the extracted factors, fuzzy inputs, numerical risk score, risk category, explanation, defensive recommendations and fuzzy membership visualisations.

Screenshots to insert before final submission

1. Main incident-analysis screen
2. Fuzzy risk result screen
3. Membership-function visualisation
4. About / methodology section
5. Deployed Streamlit application in a browser

The screenshots should be captured from the actual deployed application and kept clear enough for faculty review.

Testing

Area	Coverage
Fuzzy engine	Membership boundaries, fuzzification, rule inference, aggregation, defuzzification, score range
Validators	Pydantic validation, defaults, invalid values and edge cases
Integration	Risk pipeline with mocked LLM responses and malformed responses
Security	No hard-coded credentials; defensive-only recommendations

Limitations

LLM extraction depends on API availability and input quality. The fuzzy rule base represents an educational model rather than a universal cybersecurity standard. Scores should not replace expert security assessment. Internet access is required for live LLM calls.

Future Scope

Add CVSS-informed inputs, SIEM/log ingestion, historical incident analytics, broader expert-reviewed fuzzy rules, PDF reporting and additional LLM providers.

Conclusion

The project demonstrates a hybrid architecture in which an LLM handles natural-language understanding while a genuine fuzzy inference system computes the numerical cybersecurity risk. This separation improves transparency for an academic demonstration and keeps the final score deterministic for fixed fuzzy inputs.

References

1. Zadeh, L.A. (1965). Fuzzy Sets. *Information and Control*, 8(3), 338–353.
2. LangChain Documentation — Python documentation.
3. Google AI for Developers (2026). Gemini API / Models documentation.
4. scikit-fuzzy Documentation — Fuzzy Logic Toolbox.
5. Streamlit Documentation.
6. Pydantic Documentation.
7. Cambridge University Press (2026). *The Cambridge History of International Law — International Law in Ancient India*; related scholarship on the Arthaśāstra and state security.
8. NIST SP 800-30 Rev. 1. Guide for Conducting Risk Assessments.

Web sources: ai.google.dev and Cambridge Core pages consulted for the current model and IKS background.

GitHub Repository

[INSERT PUBLIC GITHUB REPOSITORY LINK]

Live Application

[INSERT PUBLIC STREAMLIT DEPLOYMENT LINK]

README checklist

- ✓ Project title and description
- ✓ Features and technology stack
- ✓ Installation/setup instructions
- ✓ Environment variable names only; no secret values
- ✓ Screenshots
- ✓ Live deployment link
- ✓ Student name and roll number

Final submission checks

- ✓ Public deployment opens successfully.
- ✓ GitHub repository is accessible and contains complete source code.
- ✓ Documentation is no more than 10 pages.
- ✓ IKS connection is clearly explained and sourced.
- ✓ No API keys, passwords or tokens are exposed.
- ✓ Screenshots show the actual working application.
- ✓ Student details and links are filled in before submission.

Prepared from the college's Individual IKS Project Submission Instructions. Final student-specific details and actual deployed links must be inserted before submission.